

CineCirrus

Cirrus CSS + código real do projeto

Express + Handlebars + Sequelize

O que é o Cirrus CSS?

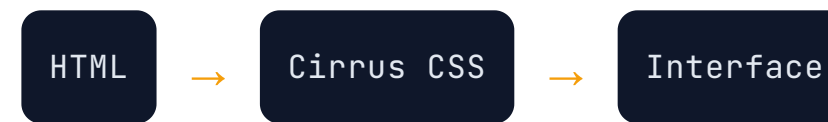
Framework CSS de interface

O Cirrus oferece classes prontas para estruturar grids, cards, tabelas, formulários, abas e outros elementos.

No CineCirrus ele entra pelo CDN no layout principal e acelera a criação da interface.

Contexto do projeto

Escolhemos o framework para cumprir a atividade com uma interface simples, responsiva e fácil de explicar.



Vantagens e desvantagens

Vantagens

- Componentes prontos reduzem o CSS manual.
- Grid responsivo facilita a adaptação para celular.
- Classes como `card`, `row` e `col` são fáceis de demonstrar.
- Boa escolha para protótipos e projetos escolares.

Desvantagens

- Projetos pequenos podem usar apenas uma parte do framework.
- O visual pronto pode exigir CSS próprio para personalização.
- Usando CDN, a interface depende do acesso à internet.
- É preciso consultar a documentação para conhecer as classes.

Seis componentes apresentados

Card

Agrupa informações de um filme ou entidade.

```
class="card"
```

Table

Organiza a tela Listar Todos.

```
class="data-table"
```

Form

Recebe dados no cadastro.

```
<form method="post">
```

Accordion

Abre e fecha uma explicação.

```
<details>
```

Tabs

Organiza conteúdos relacionados.

```
class="tab-links"
```

Modal

Destaca uma informação adicional.

```
class="modal-card"
```

A atividade pediu para não usar **button**; por isso, os comandos visuais foram feitos com links estilizados.

package.json: configuração real do projeto

Este arquivo define os comandos e dependências. O **package-lock.json** registra as versões exatas instaladas.

- **Express:** servidor e rotas.
- **Handlebars:** templates HTML.
- **Sequelize:** acesso ao banco.
- **SQLite3:** banco local.

```
"scripts": {  
  "start": "node server.js",  
  "check": "node tools/check.js"  
},  
"dependencies": {  
  "express": "^4.21.2",  
  "sequelize": "^6.37.5"  
}
```

server.js: o centro da aplicação

O servidor importa Express, Handlebars e os modelos Sequelize.

Depois define a porta, o layout padrão, a pasta views e os arquivos públicos.

É o arquivo que conecta todas as camadas.

```
const express = require('express');
const { engine } = require('express-handlebars');
const app = express();
const PORT = process.env.PORT || 3000;

app.engine('handlebars', engine({ defaultLayout:
'main' }));
app.set('view engine', 'handlebars');
app.use(express.static('public'));
```

server.js: rotas CRUD reais

```
app.get(`/${path}`, async (req, res) => {  
  const rows = await config.model.findAll();  
  res.render('entity', { isList: true, rows });  
});
```

```
app.post(`/${path}`, async (req, res) => {  
  await config.model.create(req.body);  
  res.redirect(`/${path}`);  
});
```

- **GET** busca e renderiza.
- **POST** recebe o formulário.
- `findAll()` lista registros.
- `create(req.body)` salva no banco.
- A mesma função cria rotas para as quatro entidades.

models: Sequelize + SQLite

database.js

Configura o SQLite em data/cinecirrus.sqlite usando o dialect sqlite.

Entidades

Os modelos representam as tabelas Filme, Artista, FichaTecnica e Diretor. Campos obrigatórios usam `allowNull: false`.

```
const Filme = sequelize.define('Filme', {
  titulo: { type: DataTypes.STRING, allowNull: false },
  ano: { type: DataTypes.INTEGER, allowNull: false },
  genero: { type: DataTypes.STRING, allowNull: false },
  sinopse: { type: DataTypes.TEXT, allowNull: false }
});

await sequelize.sync();
```


views: um template para três telas

main.handlebars é o layout compartilhado: menu, CSS, conteúdo e rodapé.

- **Listar Todos:** tabela com rows.
- **Detalhar:** informações de row.
- **Cadastrar:** formulário com fields.

```
{{#if isList}}  
<table class="data-table">  
  {{#each rows}}  
    <td>{{title}}</td>  
    <td>{{category}}</td>  
  {{/each}}  
</table>  
{{else}}  
<form method="post">... </form>  
{{/if}}
```

public: Cirrus + identidade visual

views/layouts/main.handlebars importa o Cirrus pela CDN:

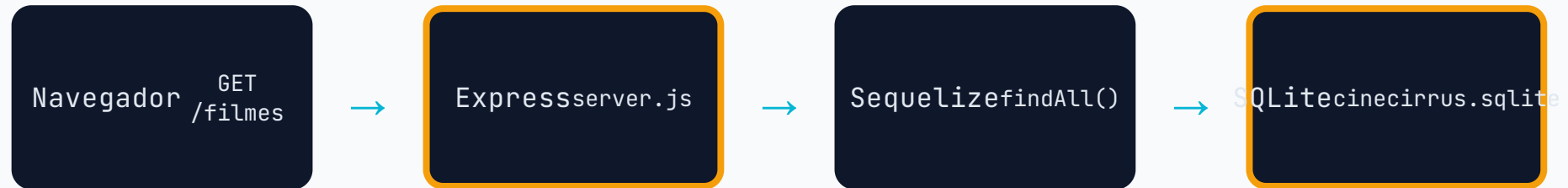
`cirrus.min.css`

`public/styles.css` complementa o framework com as cores e regras do CineCirrus.

■ azul-noite ■ ciano ■ âmbar

```
:root {  
  --navy: #10182f;  
  --blue: #0ea5e9;  
  --amber: #f59e0b;  
}  
  
.data-table {  
  width: 100%;  
  background: #fff;  
}  
  
.link-pill.primary {  
  background: var(--blue);  
}
```

Como uma página chega ao navegador



Depois da consulta, o Express entrega os dados ao Handlebars, que monta `entity.handlebars` e devolve o HTML pronto. O Cirrus CSS e o `styles.css` deixam essa página apresentável.

Para executar e apresentar

```
> npm install  
> npm run check  
> npm start
```

CineCirrus rodando em <http://localhost:3000>

O projeto une Cirrus CSS na interface, Express nas rotas, Handlebars nos templates, Sequelize na integração e SQLite no armazenamento.